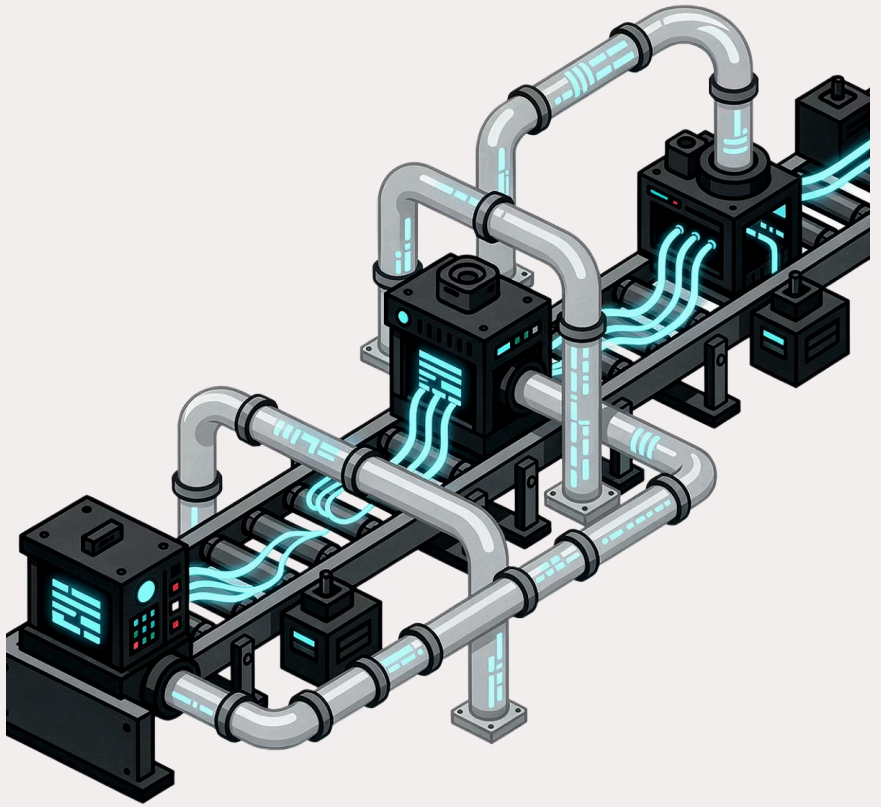


Day 7: DAG Factory & Config-Driven DAGs

Revolutionizing how teams design, deploy, and scale Apache Airflow workflows through declarative, configuration-driven development.

MODULE 1 – INTRODUCTION TO DAG FACTORY

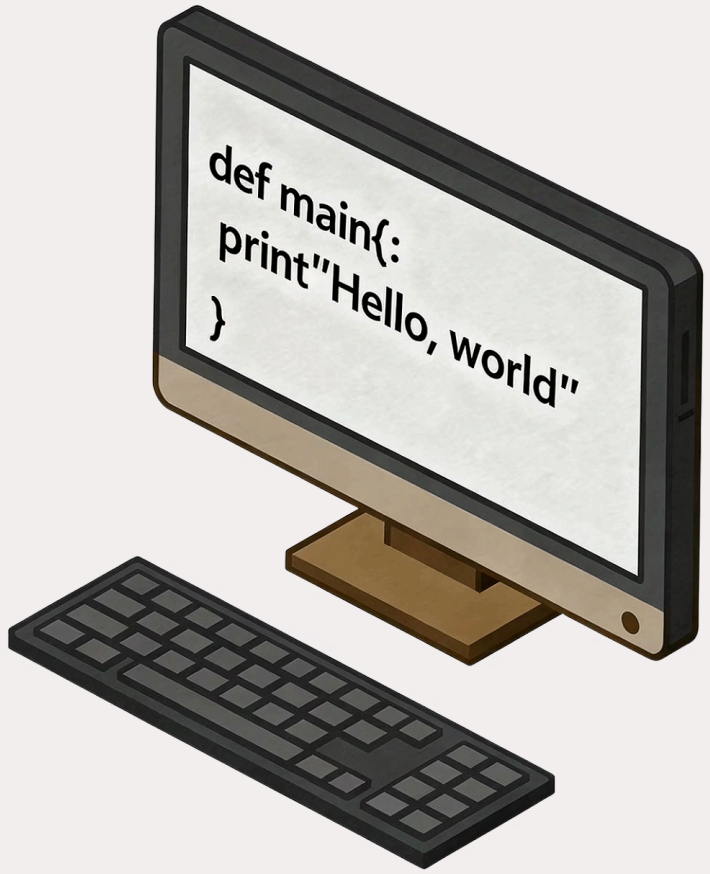
CHAPTER 1

The Traditional Airflow Landscape

Before we explore DAG Factory, let's examine how Airflow DAGs have traditionally been built — and why the status quo demanded a smarter approach.

Python-Powered DAGs: The Foundation

Apache Airflow's core strength has always been the ability to define complex workflows directly in Python. This gives engineers tremendous flexibility — conditional logic, dynamic task creation, and deep integrations are all possible. However, this power comes with a cost: meaningful Airflow development requires both solid Python expertise and a thorough understanding of Airflow's internals, making it inaccessible to analysts or data engineers who are less familiar with the framework.



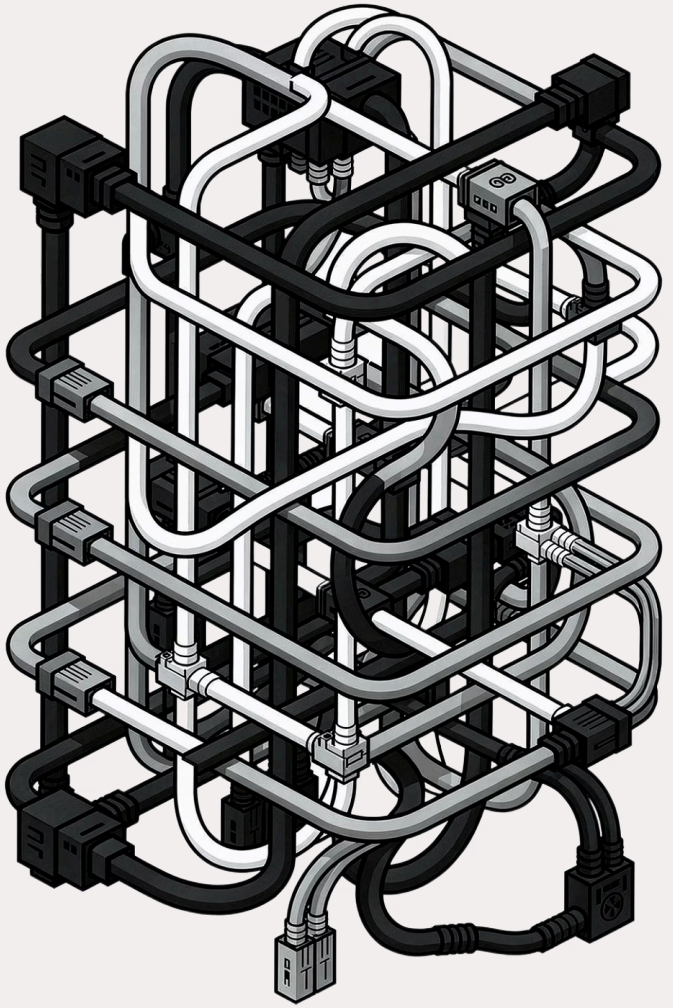
The Challenge: Scalability & Maintainability

Growing Complexity

As the number of DAGs in a project grows, the Python codebase expands rapidly. Logic becomes harder to trace, and small changes risk cascading side effects across pipelines.

Common Pain Points

- Code duplication across similar DAG definitions
- No standard structure between team members' DAGs
- Steep onboarding curve for new engineers
- Difficult to enforce organization-wide standards
- Review and auditing of pipeline logic is time-consuming



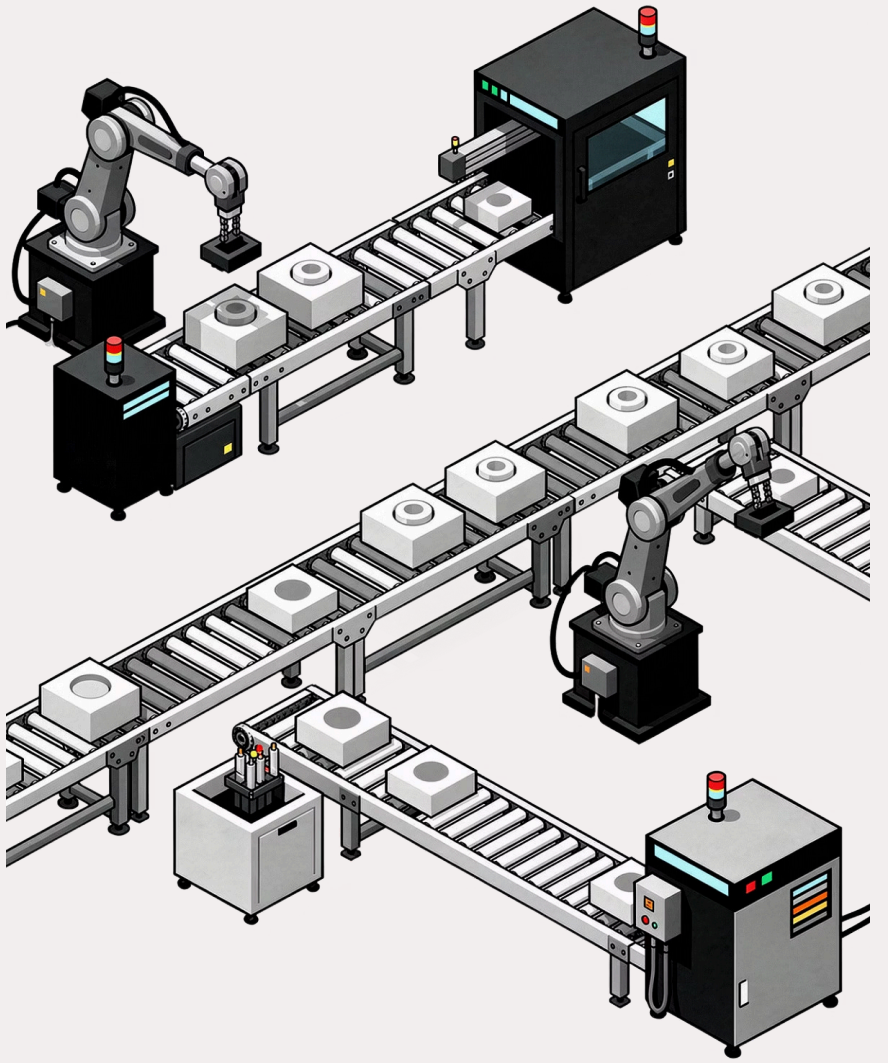
The Pythonic Maze

When every team member writes DAGs differently — each with unique patterns, variable names, and operator configurations — the codebase becomes a maze. Finding, updating, or auditing a single pipeline can take hours instead of minutes. This is the hidden cost of unstructured DAG development at scale.

CHAPTER 2

Introducing DAG Factory

A declarative approach to Airflow — where configuration replaces code and simplicity replaces complexity.



What is DAG Factory?

Open-Source Library

Developed and maintained by Astronomer, DAG Factory is a freely available Python library designed to simplify Airflow DAG creation through a declarative model.

YAML-Based Definitions

Instead of writing Python for every workflow, teams define DAG structure, tasks, schedules, and dependencies in human-readable YAML configuration files.

Automatic DAG Generation

DAG Factory parses YAML files and automatically translates them into fully functioning Apache Airflow DAG objects — no manual Python wiring required.

The Core Idea: YAML-Based DAG Definitions

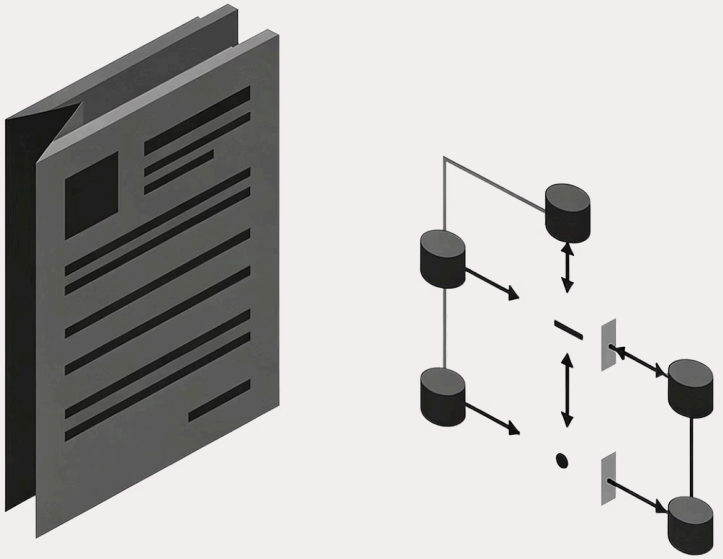
At the heart of DAG Factory is a simple but powerful concept: replace imperative Python code with declarative YAML. A single function call — `dagfactory.load_yaml_dags()` — parses your YAML files and generates complete, production-ready DAG objects registered in Airflow's global namespace.

This means your teams can define tasks, set schedules, specify operators, and declare dependencies — all without writing a single line of Python DAG logic.

Minimum requirements: **Python 3.10.0+** and **Apache Airflow 2.4+**

Key Capabilities

- Define DAG schedules and start dates in YAML
- Declare task operators and parameters declaratively
- Set upstream/downstream task dependencies
- Configure `default_args` in a single shared block
- Load entire DAG folders automatically



YAML to DAG: Simplicity Unleashed

The transformation is elegant: a well-structured YAML file on the left, a fully rendered Airflow DAG graph on the right. DAG Factory acts as the bridge, eliminating the boilerplate Python that used to sit between your intent and your workflow. The result is configurations that are readable by engineers, analysts, and reviewers alike.

Benefits of DAG Factory

Democratized Access

Construct and manage DAGs without deep Python expertise. Data analysts and ops teams can contribute directly to pipeline development.

DRY Principles

Eliminate duplicative boilerplate. Define common configurations once and reference them across all your DAGs, keeping your codebase lean and consistent.

Streamlined Maintenance

Updating a schedule, operator parameter, or retry policy across dozens of DAGs becomes a simple YAML edit rather than a multi-file Python refactor.

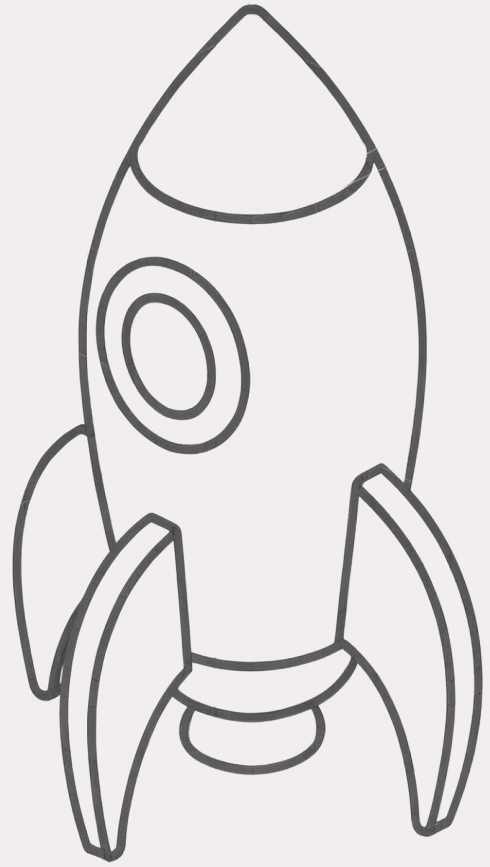
Enhanced Scalability

Onboard new pipelines in minutes. As your data platform grows, DAG Factory scales effortlessly without proportional increases in development overhead.

CHAPTER 3

Enterprise Templating & Reusability

Scaling configuration-driven DAGs across teams with consistent, reusable patterns.



Enterprise DAG Templating

What is a DAG Template?

A DAG template is a standardized YAML structure that captures a recurring workflow pattern — such as an ETL pipeline, data quality check, or reporting job — which can be reused across teams and projects with minimal modification.

Enterprise Benefits

- **Consistency:** All DAGs follow the same structural conventions, making them predictable and reviewable
- **Governance:** Platform teams control approved operator patterns and configurations
- **Lower learning curve:** New users fill in values rather than learning Airflow from scratch
- **Audit-ready:** YAML configs are easy to version, diff, and review in pull requests

Reusability Patterns in Action

Shared Configurations

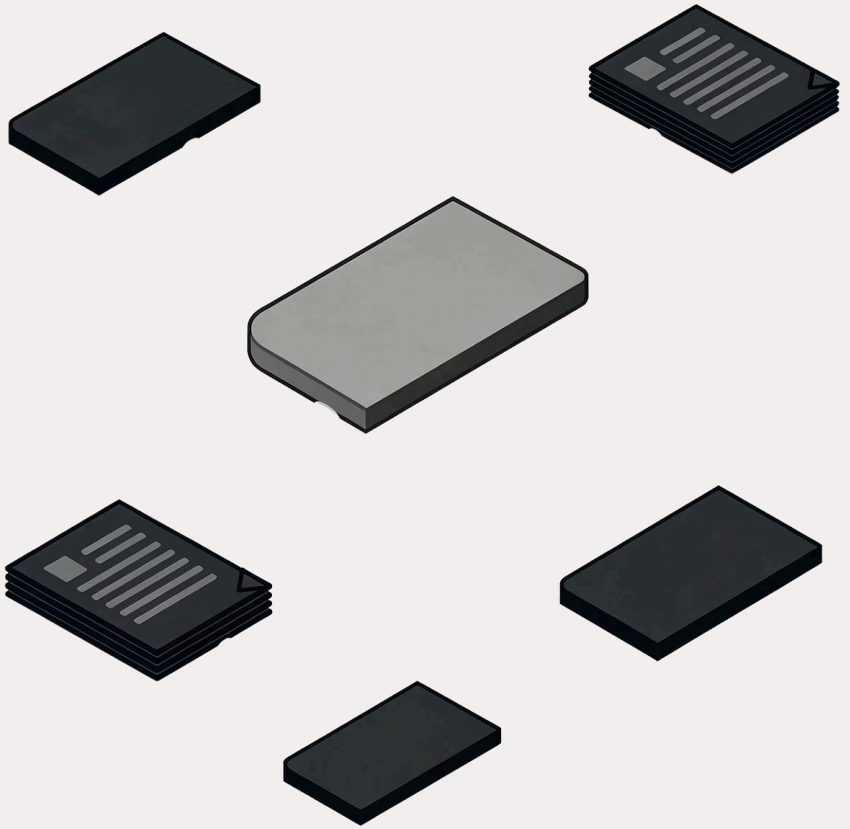
Use `default_args_config_path` or `default_args_config_dict` to define common task settings — retry counts, email alerts, timeout values — once, and apply them globally. Changes propagate instantly to all DAGs that reference them.

Modular YAML Definitions

Decompose complex workflows into smaller, independently maintained YAML files. Each file represents a logical unit that can be composed, swapped, or extended without affecting the rest of the pipeline.

Dynamic Task Mapping

Leverage Airflow's dynamic task mapping features within your YAML definitions. Generate task instances at runtime based on input configurations, enabling powerful data-driven parallelism without custom Python code.



Templating for Consistency

A single, well-designed YAML template can power dozens of unique DAGs across multiple teams. Each team customizes their values while inheriting the shared structure, ensuring organizational standards are always enforced — from task naming conventions to retry policies and alerting configurations.

Loading DAGs from a Folder

The `load_yaml_dags()` Pattern

One of DAG Factory's most powerful features is its ability to automatically discover and register every YAML DAG definition within a specified directory. Drop a new YAML file into the folder and it becomes a live Airflow DAG — no manual registration required.

```
from dagfactory import load_yaml_dags

load_yaml_dags(
  globals_dict=globals(),
  dags_folder="/path/to/your/dags"
)
```

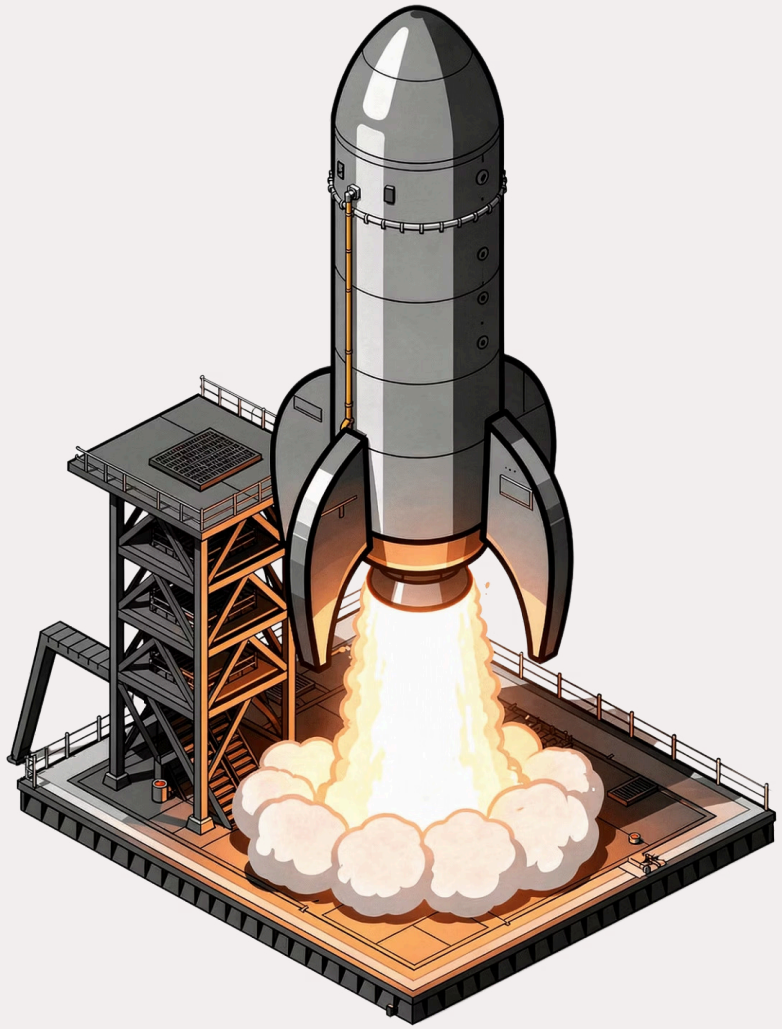
Why This Matters

- Zero-touch DAG registration for new pipelines
- Supports large-scale DAG inventories effortlessly
- Works seamlessly with CI/CD pipelines
- Integrates with Git-based DAG deployment workflows
- Teams deploy YAML files, not Python modules

CHAPTER 4

The Future of Config-Driven Workflows

Advanced features that push the boundaries of what declarative DAG development can achieve.



Beyond Basic DAGs: Advanced Features

Custom Operators

Reference and configure your own custom Airflow operators directly within YAML definitions. No need to modify Python factory code for each new operator type.

KubernetesPodOperator

Declare Kubernetes pod-based task executions entirely in YAML — including resource limits, image names, environment variables, and volume mounts.

Callbacks

Specify `on_success_callback` and `on_failure_callback` functions per task or DAG, enabling rich alerting and monitoring without boilerplate Python.

Datasets & Modern Scheduling

Leverage Airflow 2.4+ data-aware scheduling features. Define dataset dependencies and event-driven triggers declaratively within your YAML configurations.

Scalability Achieved

Imagine a single, version-controlled YAML configuration repository powering hundreds of active Airflow DAGs across multiple data domains — all consistent, all auditable, all deployable in seconds. This is the operational reality that DAG Factory enables for modern data engineering teams.

The Impact: Faster Development, Happier Teams

70%

Faster Onboarding

New team members can create production-ready DAGs without Airflow expertise, dramatically reducing ramp-up time.

5×

More Pipelines

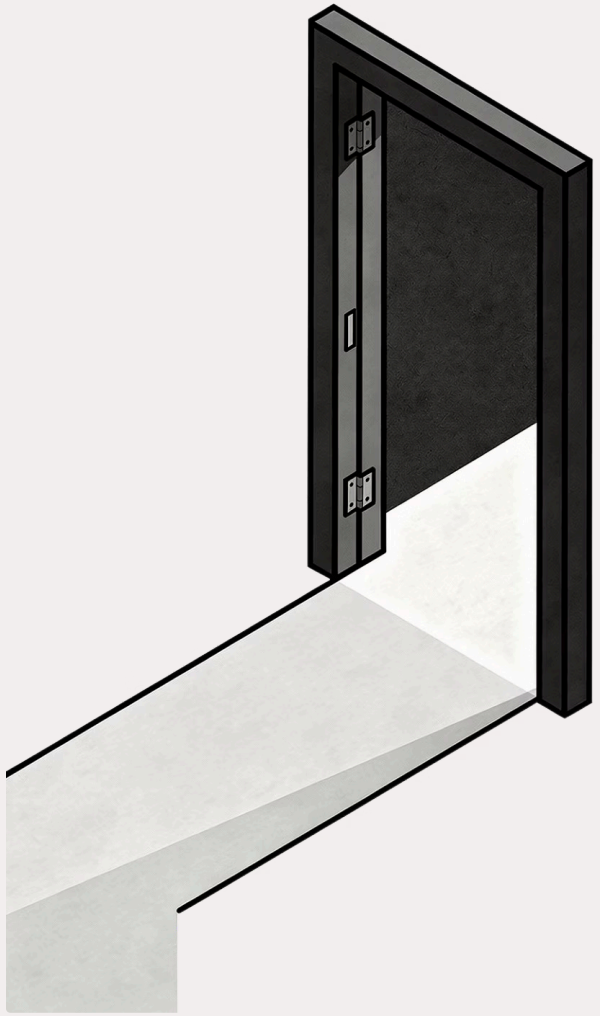
Teams report creating significantly more data pipelines in the same timeframe after adopting config-driven DAG development.

90%

Less Boilerplate

Shared templates and centralized configs eliminate the vast majority of repetitive Python code that engineers previously had to write and maintain.

The shift to declarative DAGs empowers analysts, data engineers, and platform teams to collaborate on pipeline design — fostering a culture of shared ownership over data infrastructure.



Conclusion: Embrace Declarative DAGs

DAG Factory represents a fundamental shift in how organizations build and manage Airflow workflows. By replacing Python boilerplate with human-readable YAML, teams gain speed, consistency, and accessibility — without sacrificing the power of Airflow.

Accessible

Anyone can contribute to pipeline development — not just Python experts.

Consistent

Enterprise templates enforce standards across every team and project.

Scalable

Config-driven workflows grow with your data platform, not against it.

Start building smarter, not harder. Transitioning to config-driven DAGs is one of the highest-leverage investments a modern data engineering team can make.